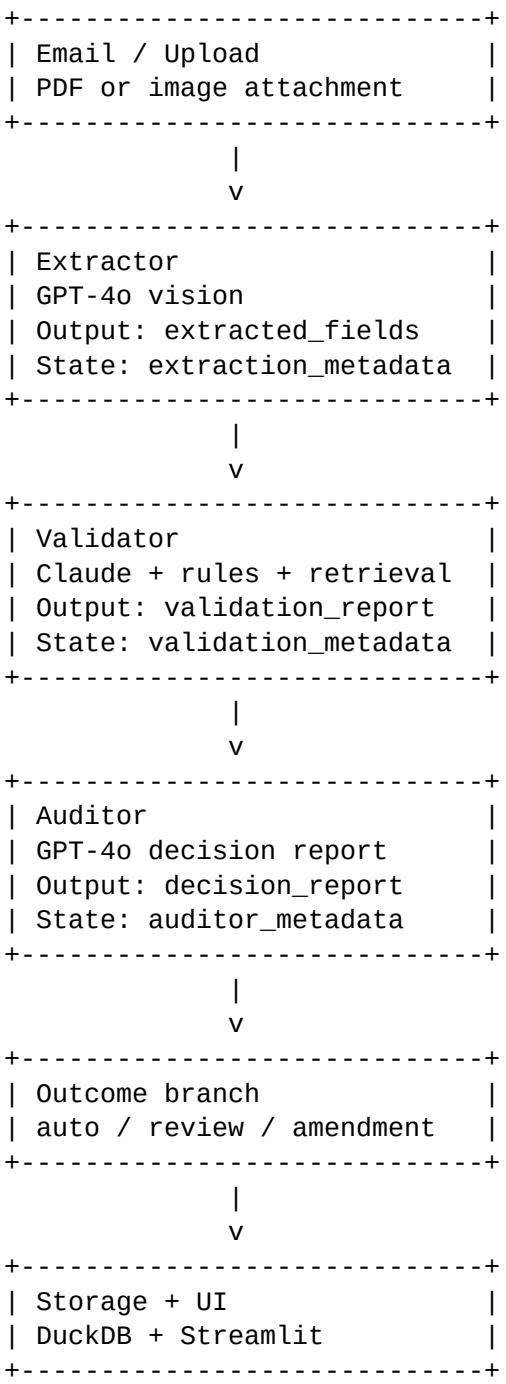
Nova Technical Writeup

Agentic Trade-Doc Automation for GoComet Nova
This writeup explains the current POC architecture, operational risks, observability plan, cost/latency profile, and what I would improve with more time.

1. Architecture Diagram

Data flow and state ownership:



PipelineState carries the live state between nodes. After every node, a JSON checkpoint is appended under data/checkpoints. Final queryable output is stored in DuckDB as extracted_json, validation_json, and decision_json.

LangGraph sequence: START -> extractor -> validator -> auditor -> outcome branch -> storage -> END.
Auditor conditional branches: auto_approve -> auto_approve_branch; flag_review -> human_review_branch; amendment -> amendment_branch. All branches persist to storage in this POC.

2. Three Nasty Failure Modes

Failure mode 1: bad or malformed documents.
Real example from testing: docs/technical_writeup.pdf was not a valid PDF before this update. ReadFile reported Invalid PDF structure. In the actual pipeline, this class of problem can happen when an email attachment is corrupt, renamed incorrectly, or scanned badly. The Extractor should fail loudly rather than invent fields. The graph records a failed checkpoint with pipeline_status=failed and the exception string, so the operator can see where the run stopped.

Failure mode 2: model/runtime dependency failure.
Real example from testing: a local smoke test could not run because the active Python interpreter was missing pydantic, and an earlier run was missing litellm. That is exactly the production risk if a worker image is built incorrectly or a provider SDK is absent. The mitigation is to compile/lint in CI, run a dependency health check at startup, and keep deterministic fallbacks in Validator and Auditor. If GPT-4o cannot write the Auditor report, the system still returns a deterministic explanation and records fallback_error in auditor_metadata.

Failure mode 3: stale orchestration names and hidden state mismatch.
Real example from testing: after renaming Router to Auditor, the graph still had routed/routing status language until we searched for stale references and cleaned it up. This is dangerous because dashboards and checkpoint recovery depend on status names. The mitigation is a strict PipelineStatus enum, rg checks for stale terms, and checkpoint entries that make every node transition visible.

3. Observability

To trace one shipment from email to verified output in production for 50 customers, every run needs a stable shipment_id/correlation_id propagated through email ingestion, LangGraph state, model calls, checkpoints, DuckDB rows, and Langfuse traces.

Trace path for one shipment:
1. Email ingestion receives attachment and creates shipment_id.
2. Extractor checkpoint records document path, extracted_fields, confidence scores, retries, and low confidence fields.
3. Validator checkpoint records field statuses, discrepancy_summary, compliance_notes, semantic checks, retrieval context, and fallback errors.
4. Auditor checkpoint records outcome, policy_path, decision_audit_report, amendment email if any, and issues considered.
5. Storage persists extracted_json, validation_json, and decision_json in DuckDB.

Production dashboard:
- Straight-through processing rate by customer and lane.
- Amendment rate and human review rate by customer.
- Pipeline failures by node: extractor, validator, auditor, storage.
- Low-confidence fields by document type and vendor.
- Model fallback/error rate by provider.
- Cost per document and token/image spend per node.

- Latency p50/p95/p99 per node.
- Top mismatch reasons and fields causing manual review.

4. Cost

Back-of-envelope POC cost per document:
- Extractor: GPT-4o vision over 1-4 rendered pages. This is the dominant cost, roughly $0.01-$0.04 depending on page count and image detail.
- Extractor retry: only low-confidence fields are retried, so normal cost is near zero, but bad scans can add another small model call.
- Validator: mostly deterministic Python. Claude is only used for semantic field checks such as goods description, roughly fractions of a cent to a few cents depending on context size.
- Auditor: one GPT-4o JSON audit report call for every case, plus one free-form amendment draft call only for amendment cases.

Expected target remains below about $0.05/document for common 1-2 page documents. It blows up when documents are long, scans are poor, every field retries, retrieval context is too large, or the Auditor is asked to summarize huge raw payloads.

Controls:
- Cap max pages rendered for the Extractor.
- Retry only low-confidence fields, never the full document unless needed.
- Keep Validator retrieval field-scoped.
- Summarize validation_report before Auditor if it grows.
- Cache known vendor layouts and port/master-data lookups.
- Track cost per node in Langfuse and alert on anomalies.

5. Latency

The slowest hop is the Extractor vision call because PDFs must be rendered to images and sent to GPT-4o vision. The second slowest hop is the Auditor if it generates both decision audit report and amendment email.

Likely latency profile:
- PDF render: sub-second to a few seconds depending on pages.
- Extractor vision call: several seconds and grows with page count.
- Field-level retry: only triggered on low confidence.
- Validator deterministic checks: fast.
- Validator semantic check: one model call for semantic fields.
- Auditor: one report call, plus one amendment email call for mismatch cases.

Fixes:
- Use OCR/template extraction for known vendor layouts and reserve vision for unknown layouts.
- Parallelize independent semantic validation calls if field count grows.
- Cache port, HS, and compliance retrieval.
- Keep Auditor prompts compact by passing discrepancy summaries rather than raw documents.
- Stream UI updates from checkpoints so operators see progress before final completion.

6. If I Had A Week Instead Of A Day

1. Add real ingestion from email/S3 with a stable shipment_id and attachment metadata.
2. Replace local JSON checkpointing with a durable store such as Postgres or MongoDB while keeping the same state contract.
3. Add a labelled eval set of 20-30 documents and score field accuracy, confidence

calibration, and routing correctness.
4. Add cross-document validation across Bill of Lading, Commercial Invoice, and Packing List.
5. Add a vendor-layout classifier so known layouts use cheap OCR/template extraction and unknown layouts fall back to GPT-4o vision.
6. Build a real rules/RAG store for customer and country compliance instead of local dummy dictionaries.
7. Add a feedback loop from CG overrides and edited amendment emails back into rules, prompts, and eval datasets.
8. Add proper CI with dependency installation, smoke tests using fake model clients, and contract tests for each agent output schema.